Atualização da foto de perfil

Etapa 1 — Manter os imports necessários para imagem

O arquivo já importa ImagePicker, Platform, AsyncStorage, router e as constantes API e IMAGEM. Não é preciso instalar dependências novas se expo-image-picker e AsyncStorage já funcionam no projeto.

```
import * as ImagePicker from "expo-image-picker";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { Platform } from "react-native";
import { API, IMAGEM } from "@/config/api";
```

Etapa 2 — Alterar o estado da foto

Substitua somente o estado imagem, que antes era string | null, e adicione o indicador de envio. O objeto ImagePickerAsset guarda uri, mimeType, fileSize e fileName.

```
const [imagem, setImagem] = useState<ImagePicker.ImagePickerAsset | null>(null);
const [enviandoFoto, setEnviandoFoto] = useState(false);
```

Etapa 3 — Criar a função enviarFoto(foto)

Insira a função abaixo dentro de PerfilScreen(), antes do return. Assim, a atualização da foto fica independente da atualização dos outros dados.

```
async function enviarFoto(foto: ImagePicker.ImagePickerAsset): Promise<boolean> {
  const tipo = foto.mimeType ?? "image/jpeg";
  const tiposAceitos = ["image/jpeg", "image/png", "image/webp"];

  if (!tiposAceitos.includes(tipo)) {
    setMensagem("Escolha uma imagem JPG, PNG ou WEBP.");
    return false;
  }
  if (foto.fileSize && foto.fileSize > 2 * 1024 * 1024) {
    setMensagem("A imagem deve ter no máximo 2 MB.");
    return false;
  }

  setEnviandoFoto(true);
  setMensagem("");
  try {
    const token = await AsyncStorage.getItem("token");
    if (!token) {
      router.replace("/");
      return false;
    }

    const form = new FormData();
    const extensao = tipo === "image/png" ? "png"
      : tipo === "image/webp" ? "webp" : "jpg";
    const nomeArquivo = foto.fileName ?? `foto.${extensao}`;

    if (Platform.OS === "web") {
      // Busca a imagem pela URI e a converte em um arquivo do tipo Blob
```

```
      const arquivo = await fetch(foto.uri).then((r) => r.blob());
      form.append("foto_cliente", arquivo, nomeArquivo);
    } else {
      // Adiciona a imagem no padrão reconhecido pelo React Native (Android e IOS)
      form.append("foto_cliente", {
        uri: foto.uri,
        name: nomeArquivo,
        type: tipo,
      } as any);
    }

    const resposta = await fetch(`${API}/cliente/foto`, {
      method: "POST",
      headers: {
        Accept: "application/json",
        Authorization: `Bearer ${token}`,
      },
      body: form,
    });
    const json = await resposta.json();

    if (!resposta.ok || !json.success || !json.data?.foto_cliente) {
      setMensagem(json.message || "Não foi possível atualizar a foto.");
      return false;
    }

    setCliente((anterior: any) => ({ ...anterior, ...json.data }));
    setMensagem("Foto atualizada com sucesso.");
    return true;
  } catch (erro) {
    console.log("Erro ao enviar foto:", erro);
    setMensagem("Não foi possível conectar ao servidor.");
    return false;
  } finally {
    setEnviandoFoto(false);
  }
}
```

Etapa 4 — Ajustando a função selecionarGaleria()

A seleção passa a confirmar automaticamente a alteração.

```
const selecionarGaleria = async () => {
  if (enviandoFoto) return;
  setModalFoto(false);
  try {
    const resultado = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ["images"],
      allowsEditing: true,
      quality: 1,
    });
    if (resultado.canceled) return;
    const foto = resultado.assets[0];
    setImagem(foto);
    await enviarFoto(foto);
    setImagem(null);
  } catch (erro) {
    console.log("Erro ao selecionar foto:", erro);
    setImagem(null);
    setMensagem("Não foi possível selecionar a imagem.");
  }
};
```

Etapa 5 — Ajustando a função tirarFoto()

Preserve a solicitação de permissão da câmera. Assim como na galeria, a captura confirma a troca e dispara o upload. Na Web, a disponibilidade da câmera depende do navegador e do dispositivo.

```
const tirarFoto = async () => {
  if (enviandoFoto) return;
  setModalFoto(false);
  try {
    if (Platform.OS !== "web") {
      const permissao = await ImagePicker.requestCameraPermissionsAsync();
      if (!permissao.granted) {
        setMensagem("É necessário permitir o acesso à câmera.");
        return;
      }
    }
    const resultado = await ImagePicker.launchCameraAsync({
      mediaTypes: ["images"],
      allowsEditing: true,
      quality: 0.7,
    });
    if (resultado.canceled) return;
    const foto = resultado.assets[0];
    setImagem(foto);
    await enviarFoto(foto);
    setImagem(null);
  } catch (erro) {
    console.log("Erro ao capturar foto:", erro);
    setImagem(null);
    setMensagem("Não foi possível abrir a câmera.");
  }
};
```

Etapa 6 — Ajustar a imagem exibida no perfil

No componente Image da foto, ajuste apenas a propriedade source.

```
source={
  imagem
    ? { uri: imagem.uri }
    : cliente?.foto_cliente
      ? { uri: `${IMAGEM}/${cliente.foto_cliente}` }
      : require("@/assets/images/img/icones/user.png")
}
```

Etapa 7 — Impedir seleção simultânea e mostrar progresso

No botão “Alterar foto”, mantenha onPress={escolherFoto} e acrescente disabled={enviandoFoto}.

```
<Pressable
  style={perfilStyle.btnAlterar}
  onPress={escolherFoto}
  disabled={enviandoFoto}
>
  <Text style={perfilStyle.txtAlterar}>
    {enviandoFoto ? "Enviando..." : "Alterar foto"}
  </Text>
</Pressable>
```

Etapa 8 — Acrecentar o return no salvamento dos dados pessoais

```
if (!resposta.ok || !json.success) {
  setMensagem(json.message || "Erro ao atualizar o perfil.");
  return;
}
```